

Technical Report

Image Rotation

Implement a function for (quality) rotation of a raster image by a given angle. The function should be implemented in several variants (shifting rows and columns, sampling a single pixel, sampling with antialiasing).

Document and demonstrate the results using examples (on several images).

Author: Samuel Kuchta

Date: June 2, 2026

Contents

1	Common Part	2
1.1	Team Composition	2
1.2	Title and Task Description	2
1.3	Description of the Target Solution	2
1.4	Data	3
1.5	Development Breakdown	3
2	Individual Part of the Author	3
2.1	Summary of Studied Knowledge and Methods	3
2.2	Planned Technologies and Methods	4
2.3	Solution Design	4
2.4	Literature and Sources Used	4
3	Project Implementation – Detailed Solution Description	5
3.1	Application Architecture	5
3.2	Rotation Algorithm – Backward Coordinate Mapping	5
3.3	Interpolation Methods	6
3.3.1	Nearest-Neighbor Interpolation	6
3.3.2	Bilinear Interpolation	6
3.3.3	Bicubic Interpolation	6
3.3.4	Lanczos Interpolation	6
3.4	Cropping Modes	7
3.5	PSNR Metric and Its Calculation	7
3.6	Graphical User Interface	7
4	Program User Guide	8
4.1	Launching the Application	8
4.2	Loading and Saving an Image	8
4.3	Rotation in the Rotator Window	8
4.4	Comparison of Methods in the Comparison Tab	8
4.5	Tips for Effective Use	9
5	Experimental Results and Evaluation	10
5.1	Experimental Methodology	10
5.2	Results	11
5.3	Agreement between Reference and Manual Implementations	12
6	Conclusion	13
6.1	Task Fulfillment	13
6.2	Benefits and Innovative Elements	13
6.3	Suggestions for Extensions	13
6.4	Summary and Recommendations for the User	13

1 Common Part

1.1 Team Composition

The project was carried out individually by a single author:

- Samuel Kuchta

1.2 Title and Task Description

Project name: Image Rotation

Task description: Implement a function for (quality) rotation of a raster image by a given angle. The function should be implemented in several variants (shifting rows and columns, sampling a single pixel, sampling with antialiasing). Document and demonstrate the results using examples (on several images).

1.3 Description of the Target Solution

The aim of the project is to design and implement a software solution enabling rotation of a raster image by an arbitrary angle. Emphasis is placed on comparing different interpolation methods in terms of output image quality and computational complexity.

The solution includes custom implementation of algorithms, their comparison with reference methods, and experimental evaluation using the objective metric PSNR (Peak Signal-to-Noise Ratio).

Specific clarifications of the task include:

- implementation of rotation around the image center,
- implementation of multiple interpolation methods (*nearest-neighbor*, *bilinear*, *bicubic*, *Lanczos*),
- design of an application allowing direct comparison of individual approaches,
- visualization of results in a graphical user interface,
- quantitative evaluation of quality using the PSNR metric,
- the possibility to experiment with different parameters and image regions.

1.4 Data

Various types of input data will be used to verify the functionality and quality of the proposed solution:

- real images in PNG, JPG, and BMP formats,
- synthetically generated patterns with high-frequency details (e.g., checkerboard, stripes),
- reference image `lena.png`, if available.

Testing will focus on:

- visual comparison of output quality,
- calculation of PSNR after a forward and inverse transformation,
- analysis of the influence of the rotation angle,
- investigation of the influence of interpolation method parameters.

1.5 Development Breakdown

Since the project was carried out individually, all development tasks are handled by a single author.

Part	Description of Activity
1	Implementation of the computational core in C++ including optimization of image rotation and integration via <code>pybind11</code> .
2	Design and implementation of the graphical user interface in <code>PyQt5</code> , including file handling and control elements.
3	Implementation of the experimental part – PSNR calculation, generation of graphs, and analysis of results.

2 Individual Part of the Author

2.1 Summary of Studied Knowledge and Methods

To solve the project, it was necessary to acquire knowledge from the field of digital image processing, in particular:

- principles of geometric transformations and image rotation,
- interpolation methods (nearest-neighbor, bilinear, bicubic, Lanczos),
- interfacing programming languages C++ and Python,
- design of graphical user interfaces,
- methods for objective image quality assessment (PSNR).

2.2 Planned Technologies and Methods

The following technologies will be used for the implementation:

- C++17 for efficient computations,
- OpenCV as a support library for image processing,
- pybind11 for integration with the Python environment,
- PyQt5 for the graphical interface,
- NumPy for numerical operations,
- Matplotlib for visualization of results,
- Pillow for input/output operations with images.

2.3 Solution Design

The proposed solution is based on the principle of backward coordinate mapping, where for each pixel of the output image its position in the original image is determined.

The implementation will allow:

- rotation of an image around its center,
- use of multiple interpolation methods,
- comparison of reference and custom implementations,
- measurement of computation time,
- quality evaluation using PSNR.

The graphical interface will provide the user with intuitive parameter control and immediate feedback in the form of result visualization.

2.4 Literature and Sources Used

The following sources were used in the project:

- OpenCV documentation,
- pybind11 documentation,
- PyQt5 documentation,
- NumPy and Matplotlib documentation.

3 Project Implementation – Detailed Solution Description

3.1 Application Architecture

The application is divided into two main parts – a computational core implemented in C++ using the OpenCV library, and a graphical user interface (GUI) created in Python with the PyQt5 framework. The connection between them is provided by the `pybind11` library, which enables direct calling of native functions from Python and efficient transfer of image data in the form of numpy arrays.

Communication proceeds as follows:

1. The user loads an image via the GUI – the loaded image is stored as a numpy array of type `uint8` with shape $(H, W, 3)$ in Python memory.
2. With each parameter change (angle, method, cropping mode, parameter a for Lanczos, sharpness parameter for bicubic), the GUI calls the corresponding C++ function, passing a reference to the numpy array and other parameters.
3. The C++ function performs the rotation and returns a new numpy array, which the GUI displays.

3.2 Rotation Algorithm – Backward Coordinate Mapping

All implemented methods (both custom and reference) use the principle of **backward mapping**. For each pixel of the output image, the position in the original image from which it should be derived is calculated.

The transformation matrix for rotation around the image center is composed of three steps:

- translation of the image center to the origin of the coordinate system (matrix T_{origin}),
- rotation around the origin by angle θ (matrix R),
- translation back to the original center (matrix T_{back}).

Resulting matrix: $M = T_{\text{back}} \cdot R \cdot T_{\text{origin}}$.

For backward mapping, the inverse matrix M^{-1} is needed, which for any point in the output image returns the position in the original image. This matrix is computed using SVD decomposition.

3.3 Interpolation Methods

3.3.1 Nearest-Neighbor Interpolation

The simplest method – the resulting pixel color is determined by the color of the nearest neighbor. The implementation uses rounding of coordinates:

```
int ix = cvRound(x_src);
int iy = cvRound(y_src);
if (ix >= 0 && ix < src.cols && iy >= 0 && iy < src.rows)
    dst.at<cv::Vec3b>(y, x) = src.at<cv::Vec3b>(iy, ix);
```

3.3.2 Bilinear Interpolation

Weighted average of the four nearest pixels. The weights are determined by the distances dx, dy :

$$f(x, y) = (1 - dx)(1 - dy)p_{00} + dx(1 - dy)p_{10} + (1 - dx)dy p_{01} + dx dy p_{11}.$$

3.3.3 Bicubic Interpolation

Bicubic interpolation uses a 4×4 pixel neighborhood and weights determined by a cubic kernel. The implementation uses the kernel shape according to OpenCV with parameter $A = -0.75$:

$$f(x) = \begin{cases} (A + 2)|x|^3 - (A + 3)|x|^2 + 1 & \text{for } 0 \leq |x| < 1 \\ A|x|^3 - 5A|x|^2 + 8A|x| - 4A & \text{for } 1 \leq |x| < 2 \\ 0 & \text{otherwise} \end{cases}$$

Parameter A (typically -0.75) influences sharpness and the amount of overshoot. In the application this parameter can be changed using a slider, allowing experimental comparison of different cubic filters (e.g., Catmull-Rom at $A = -0.5$).

3.3.4 Lanczos Interpolation

The highest quality method. The weight is given by the Lanczos kernel:

$$L(x) = \begin{cases} 1 & \text{if } x = 0 \\ \frac{a \sin(\pi x) \sin(\pi x/a)}{\pi^2 x^2} & \text{if } 0 < |x| < a \\ 0 & \text{if } |x| \geq a \end{cases}$$

Parameter a (typically 4) determines the window size $2a \times 2a$.

3.4 Cropping Modes

cut – cropping to the original size; edges exceeding the frame are cut off.

preserve – preservation of the entire rotated image; the output size is enlarged.

zoom_to_content – automatic cropping to the maximum inscribed rectangle using the function `get_max_inner_rect()`.

3.5 PSNR Metric and Its Calculation

PSNR (Peak Signal-to-Noise Ratio) is calculated according to the formula:

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{255^2}{\text{MSE}} \right), \quad \text{MSE} = \frac{1}{3HW} \sum_{i,j,c} (I_{ijc} - K_{ijc})^2.$$

In the application, PSNR is calculated between the original image and the doubly rotated image (rotation by α and then by $-\alpha$).

3.6 Graphical User Interface

The GUI contains two main tabs:

- **Rotator** – simple rotation with immediate display, setting of angle, method, cropping mode, and parameters (for Lanczos a , for bicubic sharpness A).
- **Comparison** – detailed comparison of all methods: original image with region selection, split preview, a 4×2 table with double-rotation results and PSNR, buttons for plotting PSNR graphs.

4 Program User Guide

4.1 Launching the Application

1. Install Python 3.8+, OpenCV, and a C++ compiler.
2. In the terminal, run: `python run.py`
3. The script automatically installs dependencies, checks the OpenCV path, compiles the C++ module, and launches the GUI.

4.2 Loading and Saving an Image

- **Load Image** – opens a dialog to select an image file (PNG, JPG, BMP).
- **Save Image** – saves the currently displayed rotated image.
- **Generate Checkerboard (FHD)** – creates a synthetic color checkerboard pattern 1920×1080 .

4.3 Rotation in the Rotator Window

1. Select a method, cropping mode, and optionally a parameter (a for Lanczos, A for bicubic).
2. Set the angle using the slider or by direct input.
3. The resulting image is displayed on the right, including processing time.

4.4 Comparison of Methods in the Comparison Tab

- **Make sure the cropping mode is set to CUT, otherwise PSNR will be incorrect (due to slight shifts of the image).**
- **Region of Interest Selection** – click and drag the mouse on the original image.
- **Split Preview** – directly compares the original and transformed region.
- **4×2 Table** – for each method, displays the double-rotation result, PSNR value, and a button to plot a graph.
- **PSNR Analysis** – buttons *PSNR vs Angle* (line graph) and *Boxplot of PSNR* (box plot). The graphs exclude angles with $\text{PSNR} > 1000$ (e.g., 0°) for a reasonable display of values.
- **Limit Angle Range** – restricts the angle range based on symmetry (speeds up calculations).
- **Show Manual Methods** – shows only the optimal reference solutions (speeds up calculations).

4.5 Tips for Effective Use

- For quick comparison, use the split preview and move the slider.
- When analyzing PSNR, first limit the angle range. (The function is harmonic every 90° , and from 45° - 90° mirrored, so computing the graph for more than 45° brings no practical information.)
- For the most accurate comparison, select a region with fine details (high contrast, high-frequency, and multi-colored area).

5 Experimental Results and Evaluation

5.1 Experimental Methodology

Experiments were performed on the following images:

- `lena.png` (512×512)
- Checkerboard (1920×1080)

PSNR after double rotation, the influence of parameter a (Lanczos) and parameter A (bicubic), and comparison of reference and manual implementations were evaluated.



Figure 1: Image `lena.png`, resolution 512×512 .



Figure 2: Checkerboard image, resolution 1920×1080 .

5.2 Results

PSNR (dB) after double rotation for different methods and angles over a selected region (mode cut) is shown in figures 3 and 4. Box plots summarizing the PSNR distribution across all angles are shown in figures 5 and 6.



Figure 3: PSNR vs. angle for image `lena.png`.

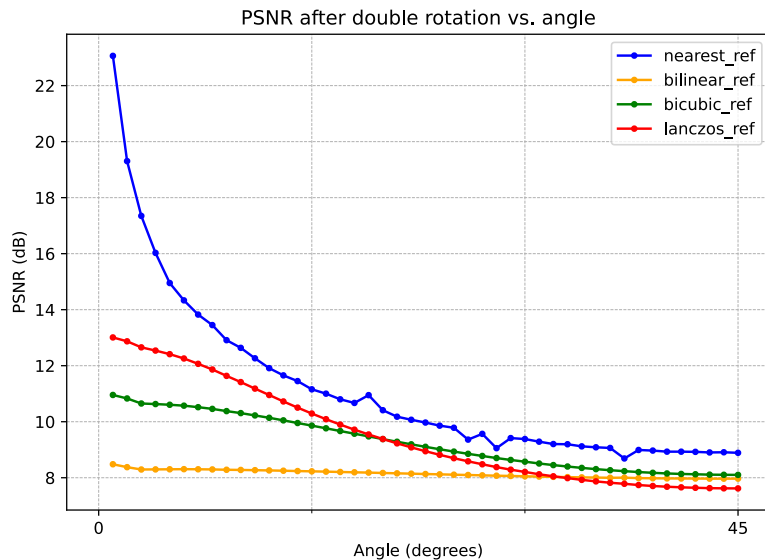


Figure 4: PSNR vs. angle for checkerboard image.

The best results for a natural image are achieved by Lanczos interpolation, followed by bicubic and bilinear. Nearest-neighbor is the worst. For the color checkerboard it is exactly the opposite, because nearest-neighbor does not create new colors (which the original image did not contain) like the other methods.

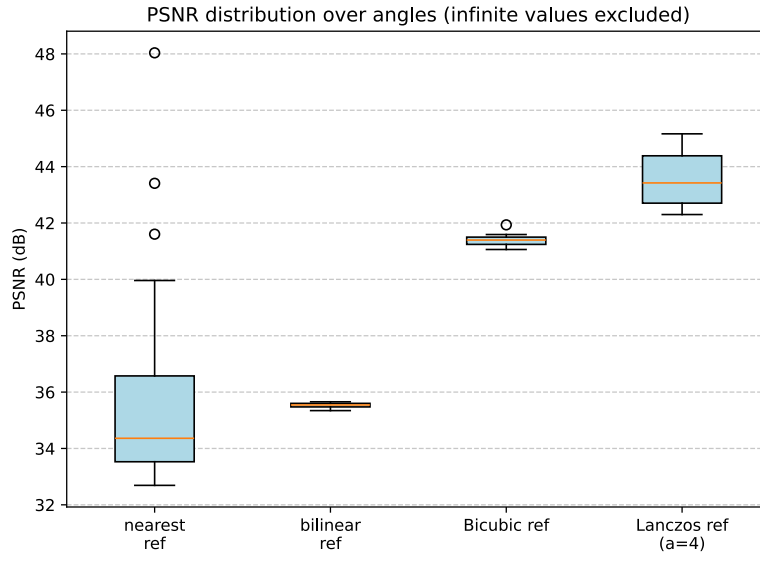


Figure 5: Box plot of PSNR for image `lena.png`.

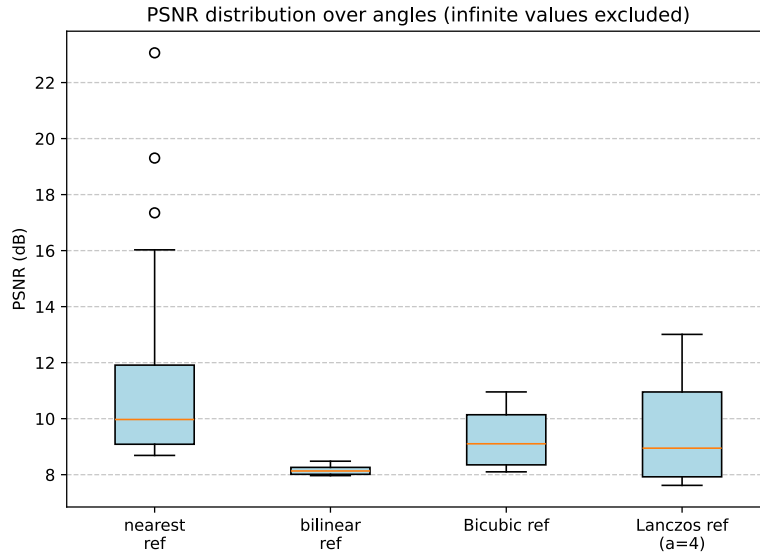


Figure 6: Box plot of PSNR for checkerboard image.

5.3 Agreement between Reference and Manual Implementations

The verified output agreement for all methods is <0.05 dB. This confirms the correctness of the custom implementations.

6 Conclusion

6.1 Task Fulfillment

A complete application for rotation of a raster image has been developed, fulfilling all points of the assignment:

- Implementation of rotation by an arbitrary angle in four variants (nearest-neighbor, bilinear, bicubic, Lanczos) – both reference and custom manual implementations.
- Comparison of quality using the PSNR metric, including selection of a region of interest.
- Demonstration on several images (user loading, checkerboard generation).
- Graphical user interface with intuitive controls and detailed analysis.

6.2 Benefits and Innovative Elements

- Split preview with direct comparison capability.
- Region of interest (ROI) for local evaluation.
- Adjustable parameters of interpolation kernels (Lanczos a , bicubic sharpness A).

6.3 Suggestions for Extensions

- Addition of other interpolation methods (BC-splines).
- Support for rotation around an arbitrary center.
- Export of PSNR results to CSV.
- Additional quantitative comparison methods besides PSNR, e.g., LPIPS.

6.4 Summary and Recommendations for the User

- General use: Lanczos_ref or Bicubic_ref interpolation (fast and high quality).
- When analyzing, use a selected region with a high level of detail.

References

- [1] OpenCV documentation – Geometric Transformations. Available online: https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html
- [2] pybind11 documentation. <https://pybind11.readthedocs.io/>
- [3] PyQt5 documentation. <https://www.riverbankcomputing.com/static/Docs/PyQt5/>
- [4] NumPy documentation. <https://numpy.org/doc/stable/>
- [5] Matplotlib documentation. <https://matplotlib.org/stable/contents.html>